



UNIVERSITY OF GENEVA

Faculty of Science

Assignment 1

Computational Imaging
14x062



Michel Jean Joseph Donnet

2026-03-14



Contents

1	Introduction	3
2	Methodology	3
2.1	Types of filters	3
2.2	Image restoration	4
2.2.1	Inverse filtering	4
2.2.2	Wiener filtering	4
2.2.3	Gradient-based image restoration	4
2.2.3.1	Stochastic gradient descent	5
2.3	Comparison metrics	5
3	Implementation	6
4	Results	6
4.1	Image filtering	6
4.2	Image restoration	9
4.3	Gradient-based image restoration	12
5	Discussion	16
5.1	Image filtering	16
5.1.1	Low-Pass filter	16
5.1.2	High-Pass filter	16
5.1.3	Inverse filter	17
5.1.4	Wiener filter	17
5.1.5	Gradient-based image restoration	17
5.1.5.1	Gradient descent	17
5.1.5.2	Stochastic gradient descent	18
6	Conclusion	19



1 Introduction

Humans have always tried to capture the present moment, both in painting and in photography. However, the photos taken are not always of good quality, that's why some computational methods, such as image filtering, have been developed to increase the quality of a given picture.

In this project, we will examine some basic image filtering methods and some basic way to reconstruct damaged pictures.

2 Methodology

The image filtering works in both spatial and Fourier domain.

The basics are the following:

- A filter is created to apply some operation on the image. For instance, a laplacian filter gives the gradient of the image.
- The filter is applied to the image in spatial or in Fourier domain.
 - In spatial domain, the convolution between the image and the filter is performed to apply the filter to each pixel of the image.
 - In Fourier domain, a simple element-wise multiplication between the image and the filter applies the filter to the image.

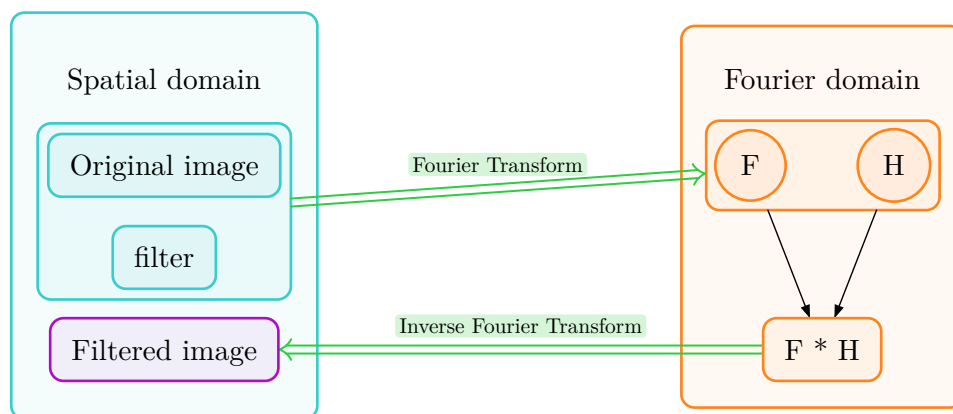


Figure 1: Image filtering in Fourier domain

2.1 Types of filters

In this project, we will use the following filters:

- Gaussian Blur: a low-pass filter that blurs the image by reducing the high-frequencies components. In this way, the small details (contained in high frequencies) are removed.
- Unsharp Mask: a high-pass filter that sharpens the image by enhancing the high-frequency components. As the low-pass filter kill the high frequencies, the unsharp mask can be obtained by substracting a blurred version of the image to the original image. Indeed,



by subtracting to the original image an image composed of low frequencies, only high frequencies are kept:

$$\text{Unsharpened image} = \text{Image} - \text{Blurred image}$$

2.2 Image restoration

2.2.1 Inverse filtering

Inverse filtering is a technique used to restore an image that has been filtered by a known filter. As the filtering operation in Fourier domain is a simple multiplication, the inverse filter consists only to inverse the multiplication to retrieve the original image. For instance, if the image was filtered in Fourier domain by a known filter H , the original image can be retrieved by multiplying the filtered image by the inverse filter $\frac{1}{H}$. Note that in general, to avoid division by zero, a small value K is added to stabilize the inverse filter:

$$\text{Inverse filter} = \frac{1}{H + K}$$

2.2.2 Wiener filtering

The wiener filter is based on the inverse filter method. However, the wiener filter takes care about the noise present in the image. Indeed, the inverse filter amplifies the noise, and the wiener filter allows to avoid the noise by adding a term dependent on the noise present in the image. The wiener filter is defined as follows:

$$\text{Wiener filter} = \frac{1}{H} \cdot \left(\frac{|H|^2}{|H|^2 + \frac{1}{\text{SNR}}} \right)$$

with $\text{SNR} = \frac{\text{Average pixel value in noised image}}{\sigma_{\text{noise}}}$. To stabilize the wiener filter, a small value K is added like for the inverse filter.

2.2.3 Gradient-based image restoration

We can consider that we have some image b that was filtered by a filter A (Here, we can consider that A is a convolution matrix). We want to retrieve the original image x such that $Ax = b$.

To do that, we can use the gradient descent method to minimize the mean squared error between Ax and b :

$$\arg \min_x f(x) = \arg \min_x \frac{1}{2} \|Ax - b\|_2^2$$

.

So the gradient descent algorithm gives us:

$$x^{k+1} = x^k - \alpha \nabla_x f(x^k)$$

The gradient $\nabla_x f$ can be defined as follows:



$$\begin{aligned}
\nabla_x f(x) &= \nabla_{x(\frac{1}{2}\|Ax-b\|_2^2)} \\
&= \nabla_x \left(\frac{1}{2}(Ax-b)(Ax-b)^T \right) \\
&= \nabla_x \left(\frac{1}{2}(x^T A^T Ax - x^T A^T b - Ax b^T + b b^T) \right) \\
&= \nabla_x \left(\frac{1}{2}(x^T A^T Ax - 2x^T A^T b + b b^T) \right) \\
&= \frac{1}{2}(2A^T Ax - 2A^T b) \\
&= A^T Ax - A^T b
\end{aligned}$$

In our work, the number of iterations is used as stopping criterion for the gradient descent algorithm.

Note that as A is a convolution matrix, the transpose of A is the convolution with the flipped version of the filter.

2.2.3.1 Stochastic gradient descent

The stochastic gradient descent is based on the gradient descent method. However, instead of computing the gradient on all the image, the stochastic gradient descent select randomly a subpart of the image to compute the gradient. In this way, the method avoids getting stuck in a local optimum where the standard gradient descent method might end up. However, since the gradient is calculated only on a portion of the image, this method will require more iterations to produce a good result.

2.3 Comparison metrics

To evaluate the difference between the original image I_{original} and modified image I_{modified} , the PSNR metric is used. The PSNR is the peak signal to noise ratio, which indicates the degree of difference between images. The PSNR is defined as in Equation 1.

$$\text{PSNR} = 10 \log_2 \left(\frac{\max(I_{\text{original}})^2}{\text{MSE}} \right) \quad (1)$$

with MSE the mean squared error between the two images defined in Equation 2.

$$\text{MSE} = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n [I_{\text{original}(i,j)} - I_{\text{modified}(i,j)}]^2 \quad (2)$$

This means that an infinite PSNR indicates that two images are identical, while a low PSNR indicates that the two images differ significantly.



3 Implementation

The code can be executed in command line and follows the documentation below:

```
TP1 of computational imaging

options:
  -h, --help          show this help message and exit
  -t, --task TASK      Enter the number of the task to execute
  -i, --image IMAGE    Path to the input image
```

To install all dependencies, the command `uv sync` can be used in `code` folder (where the file `pyproject.toml` is located). Then, the command `uv run tp1.py` automatically activate the virtual environment created and run the code.

For the gradient descent and the stochastic gradient descent, to avoid allocation of huge convolution matrix, I try to filter the image in Fourier domain. Indeed, if the image is of size 64×64 , it means that the flat version of the image is of size 4096 and then the convolution matrix needed is of size $4096 \times 4096 = 16777216$. If we consider that each element of the matrix is an integer of 1 octet, it means that the matrix uses 16 Mo. So due to memory usage, I make another implementation: we know that convolution gives bitwise multiplication in Fourier domain. So instead of working with convolution matrix of the filter with the image, I use multiplication in Fourier domain. So each pixel is updated according to the gradient of this pixel given by $\nabla_{i,j} = H(i,j)(H(i,j)F(i,j) - B(i,j))$ with H, F and B that correspond to filter, reconstructed image and damaged image in Fourier domain.

For the stochastic gradient descent, I select randomly N pixels according to the `batch_size` parameter and then I apply the gradient descent on this subset of pixels.

4 Results

4.1 Image filtering

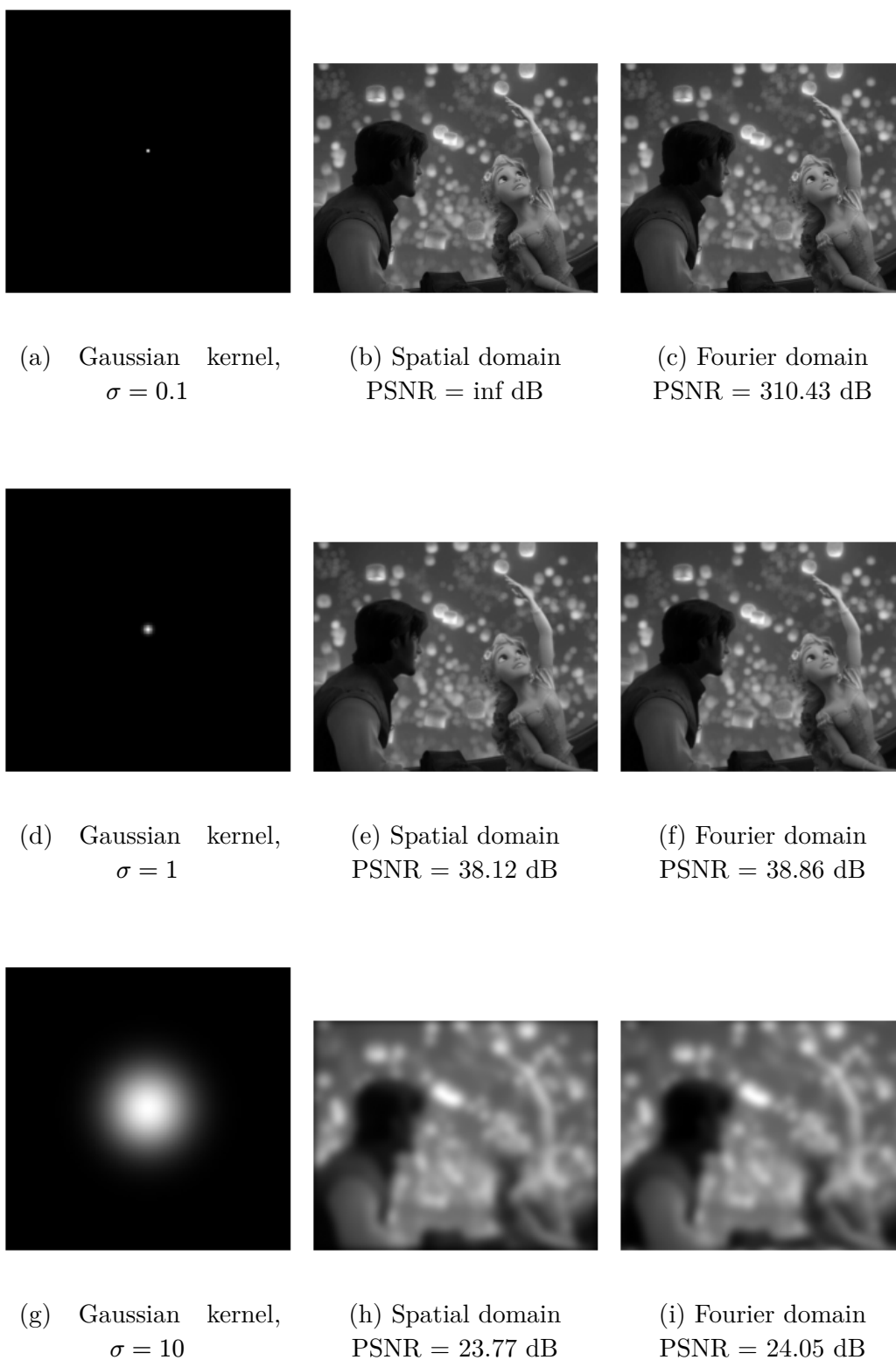


Figure 2: Low-Pass filter: Gaussian Blur

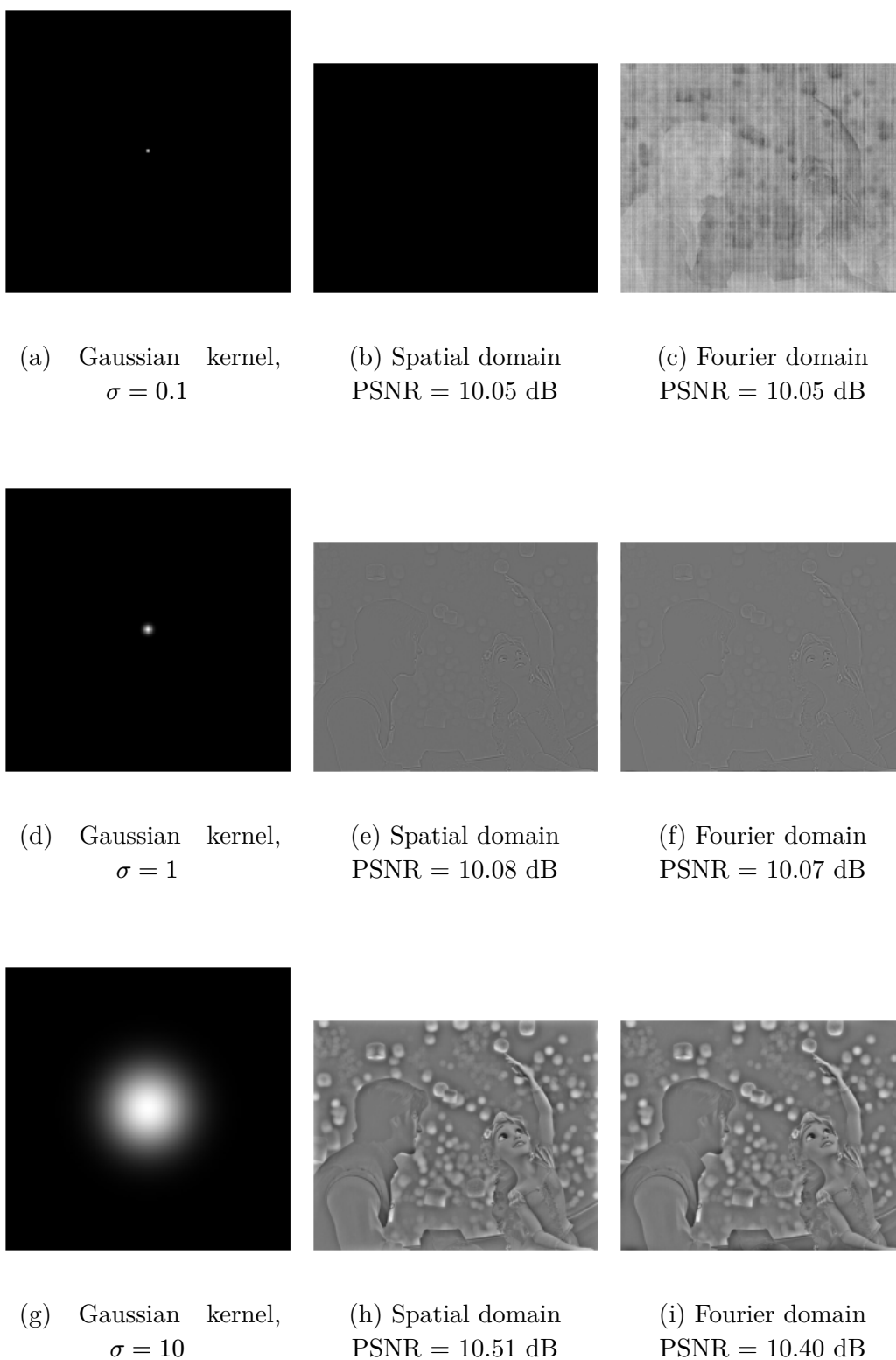


Figure 3: High-Pass filter: Unsharp Mask

4.2 Image restoration



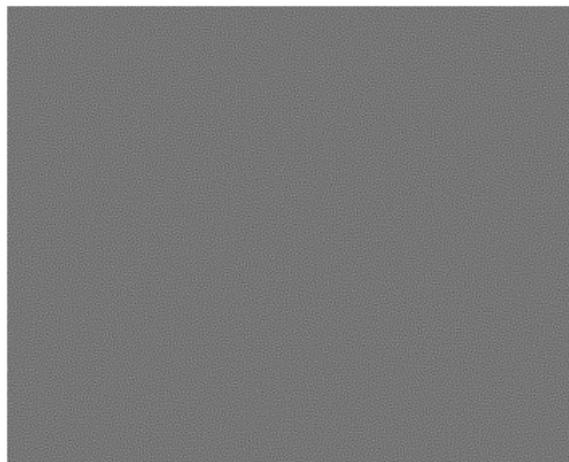
(a) Original image



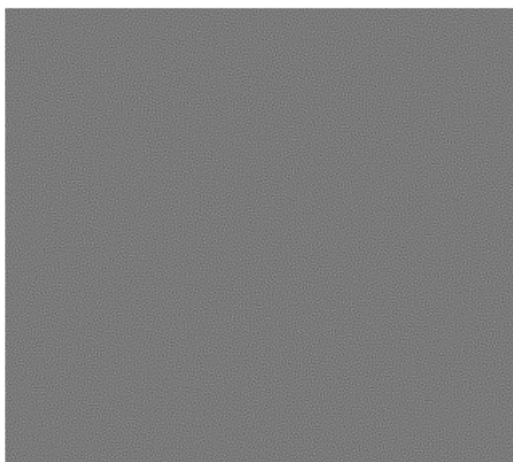
(b) blurred image
 $\text{PSNR} = 28.098427154082447$



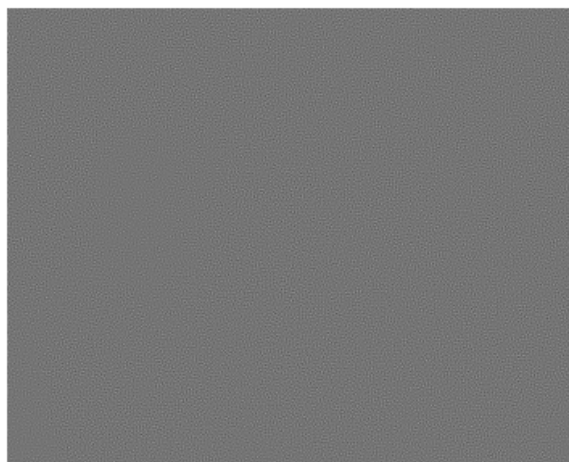
(a) Inverse filtering
noise $\sigma = 0$
PSNR = 39.31 dB



(b) Inverse filtering
noise $\sigma = 0.001$
PSNR = -139.29 dB



(c) Inverse filtering
noise $\sigma = 0.01$
PSNR = -159.29 dB



(d) Inverse filtering
noise $\sigma = 0.1$
PSNR = -178.99 dB

Figure 5: Inverse filtering on blurred image



(a) Wiener filtering
noise $\sigma = 0$
PSNR = 36.45 dB



(b) Wiener filtering
noise $\sigma = 0.001$
PSNR = 32.29 dB



(c) Wiener filtering
noise $\sigma = 0.01$
PSNR = 30.18 dB



(d) Wiener filtering
noise $\sigma = 0.1$
PSNR = 20.66 dB

Figure 6: Wiener filter on blurred image

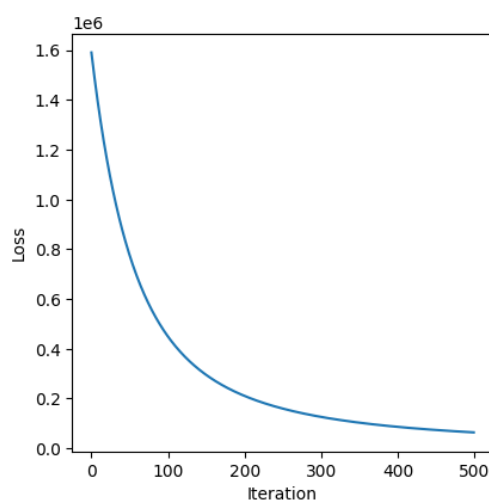
4.3 Gradient-based image restoration



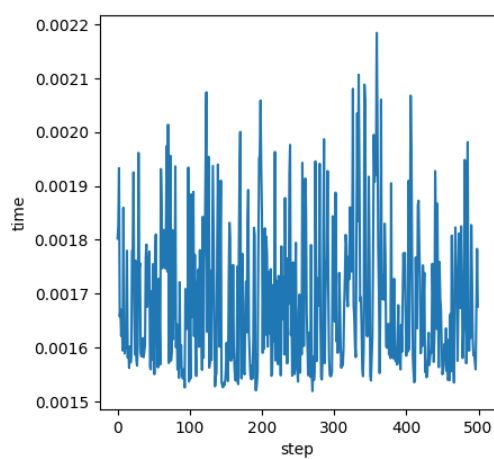
(a) Damaged image
PSNR = 26.99 dB



(b) Reconstructed image
PSNR = 29.66 dB



(c) Gradient Descent Loss



(d) Time taken for gradient descent
step

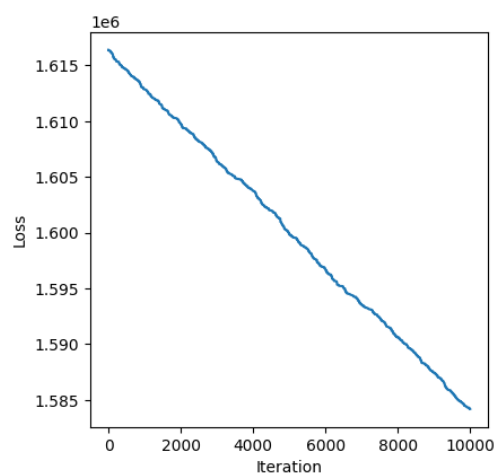
Figure 7: Gradient Descent with step size = 0.01



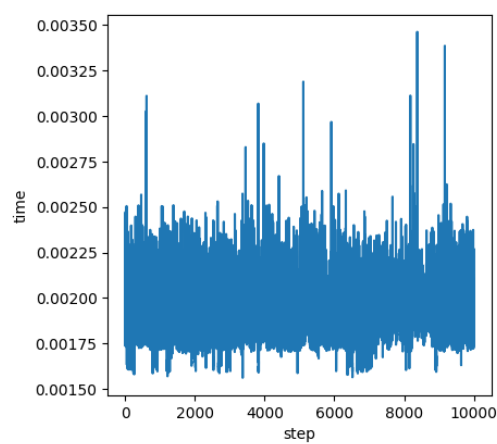
(a) Damaged image
PSNR = 26.99 dB



(b) Reconstructed image
PSNR = 27.01 dB



(c) Stochastic Gradient Descent Loss



(d) Time taken for a step

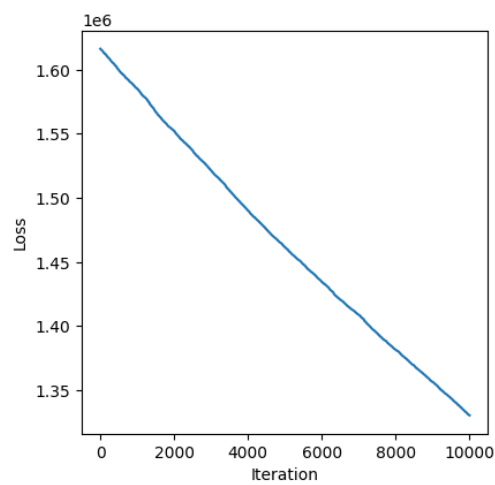
Figure 8: Stochastic Gradient Descent with batch size = 10 and step size = 0.01



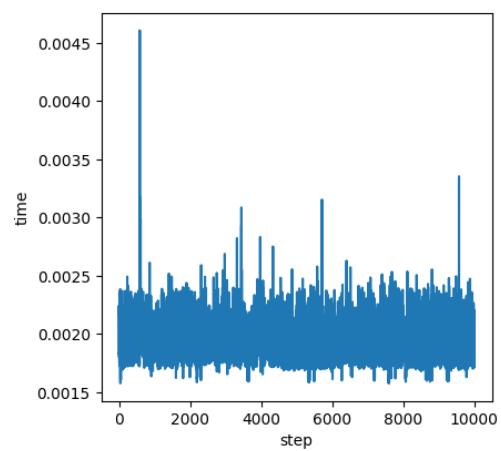
(a) Damaged image
PSNR = 26.99 dB



(b) Reconstructed image
PSNR = 27.23 dB



(c) Stochastic Gradient Descent Loss



(d) Time taken for a step

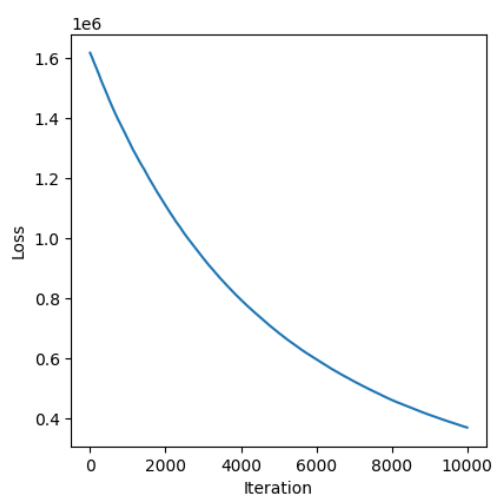
Figure 9: Stochastic Gradient Descent with batch size = 100 and step size = 0.01



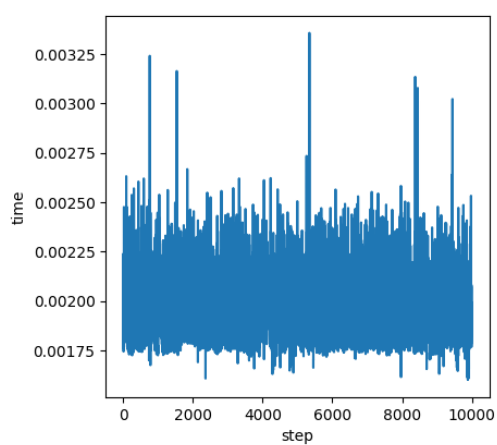
(a) Damaged image
PSNR = 26.99 dB



(b) Reconstructed image
PSNR = 28.48 dB



(c) Stochastic Gradient Descent Loss



(d) Time taken for a step

Figure 10: Stochastic Gradient Descent with batch size = 1000 and step size = 0.01



5 Discussion

5.1 Image filtering

5.1.1 Low-Pass filter

In the Figure 2, we can easily see that bigger is the Gaussian kernel, more blurred is the image. Indeed, a Gaussian kernel of 0.1 gives an image close to the original image as shown on Figure 2b and Figure 2c whereas a Gaussian kernel of 10 gives a more blurred image as shown in Figure 2h and Figure 2i. So the size of the Gaussian kernel determines the threshold to cut-off high-frequencies.

We can see that for Figure 2b and Figure 2c, the obtained PSNR is infinite in spatial domain and not infinite in Fourier domain. This is due to the size of the filter. In fact, as the σ is very small, the filter becomes close to identity filter in spatial domain. (For the identity filter, only the center of the kernel is 1, the rest is 0). In this way, the filtered image is identical to the original image, resulting in an infinite PSNR due to the definition of PSNR. Indeed, the mean squared error (MSE) between the image and itself is equal to 0.

As shown in Figure 2, the spatial and Fourier domain gives similar results. Indeed, the PSNR is always similar (around 1 dB of difference maximum...) However, if we compare Figure 2h and Figure 2i, the filtered image in spatial domain have a kind of black blurred border, which may be responsible of the 1 dB of difference between filtering in spatial and Fourier domain. This is due to the zero padding applied for the convolution of the image with the filter. Indeed, a padding is required in spatial domain to allows the application of the filter on each pixel of the image because of the convolution.

5.1.2 High-Pass filter

The results obtained by the High-Pass filter are essentially composed of edges as shown in Figure 3. For instance, in Figure 3e and Figure 3f, only the edges of the image are shown. This is due to the construction of the filter that consists of taking the original image and subtracting to it all low frequencies extracted thanks to a low-pass filter. In this way, only the high frequencies, which correspond to significant variations in the pixels, are retained, and these significant variations are found primarily along the edges.

As the sharpening of the image is done by subtracting a blurred version of the image to the original one, the results obtained in Fourier domain are very close to results obtained in spatial domain, as shown in Figure 3. Indeed, there is again around 1 dB of difference between spatial and Fourier domain.



The Figure 3b is black. This result confirm that the Gaussian kernel was too small in spatial domain, becoming the identity kernel.

The results thus show that filtering in the spatial domain is similar to that in the Fourier domain. Since there are efficient methods for transforming from the spatial domain to the Fourier domain, computational imaging favors the use of the Fourier domain for applying filters, in order to improve performance by avoiding convolution in the spatial domain.==
Image restoration

5.1.3 Inverse filter

As shown in Figure 5, the inverse filter works only when no noise is added to the image. In fact, when some noise is added to the image, the inverse filter seems to not work since only noisy gray images are produced.

So it means that the inverse filter tends to increase the noise present in the image.

But the result Figure 5a shows that when no noise is present into the image, the inverse filter works very well since the result is similar to the original image. The PSNR of the filtered image is around 39 dB whereas the PSNR of the blurred image is around 28, which means the filtered image is closer to the original image.

5.1.4 Wiener filter

Unlike the inverse filter, the Wiener filter is less sensitive to noise. In fact, when noise is added, the image remains fairly close to the original: the result is not simply a gray noise image as shown in Figure 6b, Figure 6c and Figure 6d. This behavior can be explained by the structure of the Wiener filter. In fact, the filter considers noise through the added term that depends on the noise ($\frac{1}{\text{SNR}}$). However, the filter construction is still sensitive to noise since obtained images are blurred with a decreasing PSNR when the noise is increased.

And unlike the inverse filter, the image obtained without added noise is of lower quality: it has a PSNR of 36 dB, compared to 39 dB for the inverse filter.

5.1.5 Gradient-based image restoration

5.1.5.1 Gradient descent

Why using the gradient descent ? The gradient descent is useful especially when the filter is not invertible.

As shown on Figure 7, the gradient descent algorithm seems to work: the obtained image is visually better than the damaged image. And the PSNR reflects this difference, as the reconstructed image has a higher PSNR than the damaged image, which means that the reconstructed image is closer to the original image.



Furthermore, the loss associated with the gradient descent method follows an exponential decay as the number of iterations increases, like shown on Figure 7c. It means that to have a good result, the computational power required increase exponentially.

However, the execution time for each step seems to be constant, as shown on Figure 7d.

5.1.5.2 Stochastic gradient descent

The choice of initial parameters is critical in the stochastic gradient descent method for finding the right balance between computation time and the quality of the results.

Indeed, with a batch size of 10, the method improves image quality by only 0.2 dB; with a batch size of 100, the improvement is 0.3 dB; and with a batch size of 1000, it reaches 1.5 dB.

These results are primarily due to the batch size settings: since we are sampling only 10, 100, and 1000 pixels, respectively, from an image of approximately $250 \times 250 = 62500$ pixels, this sample is negligible compared to the entire image and represents at most 2% of it.

However, with more iterations, the results might be better. In fact, the loss obtained using the stochastic gradient descent method appears to follow an exponential decline, as shown in Figure 10c, but in the case of Figure 8c, the loss appears linear because the number of iterations is insufficient, meaning that each iteration improves the final result by the same amount: more iterations would have been needed to observe an exponential decline.

Thanks to the subpart selected, the execution time for a single step is theoretically shorter than with the traditional gradient descent method, which calculates the gradient of the entire image. But due to the size of the image, this time is negligible. In fact, while the gradient is calculated only on a portion of the image, a list of random position indices must be generated, which tends to offset the time savings achieved through the shorter gradient calculation due to the small size of the image.

The stochastic gradient descent method thus prevents gradient descent from getting stuck in a local optimum, but requires more parameters to be tuned in order to achieve a good balance between the quality of the results and computational power.

6 Conclusion

We have seen, then, that filters can be applied in both the spatial domain and the Fourier domain. In image processing, filtering in the Fourier domain is preferred because of its computational efficiency.

Furthermore, a filter applied to an image can be inverted to recover the original image. However, this inversion is highly sensitive to noise. This is why certain filters, such as Wiener filter, takes noise into account to reduce this sensitivity while maintaining the efficiency needed to recover the original image.

Sometimes, the filter cannot be inverted. Methods such as gradient descent or stochastic gradient descent then allow, through iterative processing, the recovery of the original image.

Stochastic gradient descent is an improvement on gradient descent designed to prevent it from getting stuck in a local optimum. However, it introduces additional parameters to be optimized in order to achieve the best results with the lowest possible computational power.

We may wonder whether image filtering can also be performed in the wavelet domain.